

Solución con Divide y Vencerás

Idea General

La técnica Divide y Vencerás consiste en resolver un problema grande separándolo en subproblemas más pequeños e independientes.

El proceso se divide en tres fases:

1. Dividir el problema.
2. Resolver cada parte recursivamente.
3. Combinar las soluciones parciales.

División del Problema

Si el sistema recibe:

12 tareas

el algoritmo las divide en:

Grupo 1 → 6 tareas

Grupo 2 → 6 tareas

Luego:

- cada grupo se resuelve individualmente.
- y finalmente las cargas se combinan.

Fases del Algoritmo

Dividir

El algoritmo divide la lista de tareas en dos mitades:

```
mitad = len(tareas) // 2
```

Vencer

Cada mitad se resuelve recursivamente utilizando el mismo algoritmo.

Esto permite reducir progresivamente el tamaño del problema.

Combinar

Las cargas parciales de cada subproblema se suman para obtener la carga final de los servidores.

Funcionamiento General

El algoritmo:

- divide las tareas.
- resuelve cada subconjunto.
- asigna tareas a los servidores.
- y combina resultados parciales.

Aunque no explora todas las combinaciones posibles como fuerza bruta, logra una solución mucho más rápida.

Análisis de Complejidad

Relación de Recurrencia

El problema se divide en dos partes aproximadamente iguales:

$$T(n) = 2T(n/2) + O(n)$$

Donde:

- $2T(n/2)$ representa las llamadas recursivas.
- y $O(n)$ corresponde al proceso de combinación.

Aplicación del Teorema Maestro

Para:

$$T(n) = aT(n/b) + f(n)$$

se tiene:

- $a = 2$
- $b = 2$
- $f(n) = O(n)$

resultado:

$O(n \log n)$

Complejidad Espacial

El algoritmo utiliza:

- llamadas recursivas.
- estructuras auxiliares.
- y listas temporales.

Por lo tanto:

$O(n)$

Comparativa Final

Técnica	Complejidad Temporal	Complejidad Espacial	¿Solución óptima?	Tiempo real (n=100)	Tiempo real (n=1000)
Fuerza Bruta	$O(m^n)$	$O(n)$	Sí	Muy alto	Inviabile
Recursividad	$O(m^n)$	$O(n)$	Sí	Muy alto	Inviabile
Greedy	$O(n^2)$	$O(n)$	No siempre	Muy bajo	Bajo
Backtracking	$O(m^n)$ peor caso	$O(n)$	Sí	Medio	Alto
Divide y Vencerás	$O(n \log n)$	$O(n)$	No siempre	Bajo	Muy bajo

- Fuerza Bruta y Recursividad presentan un crecimiento exponencial.
- Greedy es extremadamente rápido.
- Backtracking mejora considerablemente el rendimiento mediante podas.
- Divide y Vencerás logra un crecimiento mucho más eficiente.

Si se necesita la mejor solución posible

La mejor opción es:

Backtracking

Porque:

- encuentra soluciones óptimas.
- reduce búsquedas innecesarias.

- y mantiene resultados precisos.

Si se necesita velocidad

La mejor opción es:

Greedy o Divide y Vencerás

Porque:

- son mucho más rápidos.
- manejan grandes cantidades de tareas.
- y consumen menos tiempo de procesamiento.

Técnica	Ventaja principal	Desventaja principal
Fuerza Bruta	Garantiza solución óptima	Muy lenta
Recursividad	Representa bien el árbol de decisiones	Alto costo computacional
Greedy	Muy rápida	No siempre óptima
Backtracking	Reduce búsquedas inútiles	Más compleja
Divide y Vencerás	Muy eficiente	No garantiza optimalidad

Conclusiones

Durante el desarrollo del proyecto se observó cómo un mismo problema puede resolverse utilizando diferentes paradigmas algorítmicos, cada uno con ventajas y desventajas particulares.

Cada técnica aborda el problema desde una perspectiva distinta:

- algunas priorizan exactitud.
- otras eficiencia.
- y otras simplicidad de implementación.

Técnica	Uso recomendado
Fuerza Bruta	Problemas pequeños o validación
Recursividad	Árboles, grafos, exploración de estados
Greedy	Sistemas en tiempo real
Backtracking	Problemas con restricciones complejas
Divide y Vencerás	Grandes volúmenes de datos

Aprendizajes Obtenidos

Fuerza Bruta

Se aprendió que:

explorar todas las soluciones garantiza encontrar el resultado óptimo, pero el costo computacional crece exponencialmente.

Esto demuestra las limitaciones de los algoritmos exhaustivos en problemas grandes.

Recursividad

La recursividad permitió representar el problema como un árbol de decisiones.

Además:

facilitó la comprensión de cómo se generan las soluciones, y sirvió como base para implementar backtracking posteriormente.

Greedy

El algoritmo greedy mostró cómo decisiones locales rápidas pueden generar soluciones eficientes en poco tiempo.

Sin embargo:

también se observó que rapidez no siempre implica optimalidad.

Backtracking

Backtracking permitió mejorar significativamente la recursividad mediante podas.

Se aprendió que:

eliminar ramas innecesarias puede reducir enormemente el tiempo de ejecución, manteniendo soluciones óptimas

.

Divide y Vencerás

La técnica Divide y Vencerás mostró cómo dividir un problema grande en partes pequeñas puede mejorar considerablemente el rendimiento.

También permitió aplicar el Teorema Maestro para analizar complejidad.

Limitaciones Encontradas

Durante el desarrollo se encontraron varias limitaciones:

- crecimiento exponencial en fuerza bruta.
- alto consumo de memoria en recursividad.
- falta de optimalidad en greedy.
- complejidad de implementación en backtracking.
- y dificultad para combinar soluciones en divide y vencerás.